

Day 8-9 Lecture Notes: Choosing the Right AI Approach & Designing for Reality

NLP vs Classical ML vs LLM + Cost & Architecture Trade-offs

Slide 1: Title - Introduction to the Session

Topic: Day 8–9: Choosing the Right AI Approach & Designing for Reality

This is a crucial transition point in your AI learning journey. We're moving from being **model builders** to becoming **AI solution designers**. This means you'll learn not just how to build models, but how to decide which approach to use and how to deploy it successfully in real-world scenarios.

Slide 2: Why This Session Matters

What You've Learned So Far:

- **Training and evaluating models:** You know how to build ML models and assess their performance using metrics
- **Preventing overfitting and leakage:** You understand how to avoid common pitfalls that make models fail
- **Selecting models intelligently:** You can choose appropriate algorithms for different problems

The Critical Next Step:

Now comes the **hardest part** of AI work: **deciding what to build and how to deploy it in the real world.**

Important Reality Check: Most AI projects don't fail because of bad modeling techniques. They fail because:

- The wrong approach was chosen for the problem
- Real-world constraints weren't considered (cost, speed, infrastructure)
- The solution wasn't designed with production in mind

Example: A company might build a highly accurate image recognition model, but if it takes 30 seconds to process one image and costs \$5 per prediction, it's useless for a real-time mobile app that needs instant results.

Slide 3: The Core Question - Framework for Every AI Decision

Before writing a single line of code, ask these three fundamental questions:

1. Is this even an AI problem?

Not every problem needs AI. Sometimes simpler solutions work better.

Example:

-  Bad: Using AI to check if a password has 8 characters
-  Good: Using a simple rule (if length >= 8)

2. If yes, what kind of AI?

There are different types: Rules, Classical ML, NLP, LLMs. Each has its place.

3. What are the constraints?

Consider:

- **Budget:** How much can we spend?

- **Speed:** How fast must results be?
- **Data:** What data do we have?
- **Accuracy requirements:** How perfect must it be?

Critical Warning: Getting these answers wrong leads to wasted months and money. This is why we spend two full days on this topic.

PART 1: NLP vs CLASSICAL ML vs LLM (Day 8 Content)

Slide 4: First Decision - Rules vs Learning

The Golden Rule:

Always ask first: Can this be solved with rules?

When Rules Work Best:

Rules are perfect when three conditions are met:

1. **Logic is clear and straightforward**
 - The decision process can be written as "if-then" statements
 - No ambiguity in how to make decisions
2. **Rules are stable over time**
 - The logic won't change frequently
 - No need for continuous adaptation
3. **Edge cases are limited**
 - Few exceptions to the rules
 - Easy to handle special situations

Examples of Rule-Based Solutions:

Example 1: Password Validation

If password length < 8: reject

If password has no numbers: reject

If password has no special characters: reject

Otherwise: accept

This is perfect for rules because:

- Logic is crystal clear
- Requirements don't change often
- All edge cases are known

Example 2: Tax Calculation

If income < \$10,000: tax = 0%

If income \$10,000-\$40,000: tax = 10%

If income > \$40,000: tax = 20%

Rules work because tax brackets are defined by law and stable.

When to Move Beyond Rules:

Signal to use ML: When rules "explode" in complexity

- You have hundreds of if-else statements
- Rules have many exceptions
- Logic becomes impossible to maintain

Example of Rule Explosion: Imagine detecting spam emails:

- If email contains "free": maybe spam
- But not if from trusted sender
- But yes if from .xyz domain
- Unless it mentions known company
- But not if user previously clicked similar emails
- ... (rules keep growing!)

At this point, **Machine Learning** becomes the better choice.

Slide 5: Classical Machine Learning

What is Classical ML?

Classical ML refers to traditional machine learning algorithms that learn patterns from structured data. These are the "workhorse" models of data science.

When Classical ML Works Best:

Three key conditions:

1. **Data is structured (tables/spreadsheets)**
 - Data fits into rows and columns
 - Clear features (age, income, previous purchases, etc.)
 - Think Excel spreadsheets or databases
2. **Clear target variable exists**
 - You know what you're trying to predict
 - You have labeled examples
 - Input → Output relationship is clear
3. **Problem is classification or regression**
 - **Classification:** Predicting categories (yes/no, fraud/legitimate, A/B/C)
 - **Regression:** Predicting numbers (price, temperature, count)

Real-World Examples:

Example 1: Credit Risk Scoring

- **Input features:** Income, credit history, debt ratio, employment duration
- **Output:** High risk / Medium risk / Low risk
- **Why Classical ML?**
 - Structured data (all numbers and categories)
 - Clear target (risk level)
 - Historical data available
 - Fast predictions needed
 - Explainability required for regulations

Example 2: House Price Prediction

- **Input features:** Square footage, bedrooms, location, age
- **Output:** Price in dollars
- **Why Classical ML?**
 - All features are structured
 - Regression problem (predicting a number)
 - Fast and cheap predictions

- Easy to explain to customers

Example 3: Customer Churn Prediction

- **Input features:** Usage frequency, support tickets, contract length
- **Output:** Will churn (yes/no)
- **Why Classical ML?**
 - Structured customer data
 - Binary classification
 - Need to process millions of customers quickly

Common Classical ML Models:

Linear Models:

- **Linear Regression:** For predicting continuous values
- **Logistic Regression:** For binary classification (yes/no)
- Simple, fast, interpretable

Tree-Based Models:

- **Decision Trees:** Like flowcharts that make decisions
- **Random Forest:** Many trees voting together
- **XGBoost/LightGBM:** Advanced tree methods
- More powerful, handle complex patterns

Key Advantages:

- ✓ Fast training and prediction
 - ✓ Work with small datasets
 - ✓ Interpretable results
 - ✓ Low cost
 - ✓ Proven reliability
-

Slide 6: Natural Language Processing (Traditional NLP)

What is Traditional NLP?

Traditional NLP refers to rule-based and statistical methods for working with text data **before** the deep learning revolution. These are lighter-weight approaches than modern LLMs.

What Traditional NLP Focuses On:

Three main areas:

1. **Text Classification**
 - Categorizing text into predefined categories
 - Example: "Is this email spam or not spam?"
2. **Information Extraction**
 - Pulling specific information from text
 - Example: Finding all dates, names, or prices in documents
3. **Search and Ranking**
 - Finding relevant documents
 - Sorting results by relevance
 - Example: Search engines before AI

Key Techniques:

1. TF-IDF (Term Frequency-Inverse Document Frequency)

- Measures how important a word is to a document
- Common words (the, is, and) get low scores
- Rare but meaningful words get high scores

Example: Document: "The cat sat on the mat"

- "the" appears often everywhere → low TF-IDF
- "cat" is rare → high TF-IDF
- Use these scores to understand document topics

2. Bag of Words

- Treats text as a collection of words
- Ignores grammar and word order
- Just counts word frequencies

Example: "I love pizza. Pizza is great!" → {I: 1, love: 1, pizza: 2, is: 1, great: 1}

3. Word Embeddings

- Represents words as numbers (vectors)
- Similar words have similar numbers
- Example: "king" and "queen" are close in vector space

Real-World Examples:

Example 1: Sentiment Analysis

- **Task:** Is this product review positive or negative?
- **Approach:** Count positive words (great, excellent) vs negative words (bad, terrible)
- **Why Traditional NLP?**
 - Simple classification task
 - Fast processing needed
 - High accuracy achievable with simple methods

Example 2: Spam Filtering

- **Task:** Classify emails as spam or not spam
- **Approach:** Look for spam keywords, suspicious patterns
- **Why Traditional NLP?**
 - Well-defined problem
 - Fast processing (millions of emails daily)
 - Low cost requirement
 - Proven effectiveness

Reference Source: Stanford NLP IR Book (<https://nlp.stanford.edu/IR-book/>)

When to Use Traditional NLP:

- Text problem but with limited scope
 - Classification with clear categories
 - Speed and cost are important
 - Don't need deep understanding of context
-

Slide 7: Large Language Models (LLMs)

What are LLMs?

LLMs (Large Language Models) are **deep learning models trained on massive amounts of text data** from the internet, books, and other sources. They represent the cutting edge of AI language understanding.

Key Characteristics:

- **"Large"**: Billions of parameters (think of parameters as the model's "knowledge")
- **"Language"**: Specialized in understanding and generating human text
- **"Models"**: Mathematical systems that learn patterns

Popular Examples: GPT-4, Claude, Gemini, Llama

What LLMs Excel At:

1. Natural Language Understanding

- Understand context, nuance, and implicit meaning
- Can handle ambiguous questions
- Understand references and connections

Example: Question: "What's the capital of the country where the Eiffel Tower is located?"

- LLM understands: "Eiffel Tower" → Paris → France → Capital of France → Answer: Paris
- Traditional NLP would struggle with this multi-step reasoning

2. Text Generation

- Write coherent, contextual text
- Maintain style and tone
- Generate creative content

Example: Prompt: "Write a professional email apologizing for a delayed shipment"

- LLM generates: "Dear [Customer], We sincerely apologize for the delay in your recent order..."

3. Generalization Across Tasks

- One model handles many different tasks
- No need to retrain for each new task
- Learns from few examples (few-shot learning)

Example: Same LLM can:

- Translate languages
- Summarize documents
- Answer questions
- Write code
- Extract information

Real-World Examples:

Example 1: Chatbots / Conversational AI

- **Use Case:** Customer service assistant
- **Why LLM?**
 - Handles unpredictable questions
 - Maintains context across conversation
 - Generates natural responses

- No need to script every possible conversation

Example 2: Document Q&A

- **Use Case:** "Analyze this 50-page legal contract and tell me about termination clauses"
- **Why LLM?**
 - Understands complex legal language
 - Can reason across many pages
 - Extracts relevant information intelligently

Example 3: Summarization

- **Use Case:** Summarize 100 customer reviews into key themes
- **Why LLM?**
 - Understands context and meaning
 - Identifies patterns across texts
 - Generates coherent summaries

Reference Source: Hugging Face NLP Course (<https://huggingface.co/learn/nlp-course>)

The Power of LLMs:

- Deep contextual understanding
- Flexible and adaptable
- Handle complexity that stumps other approaches

The Cost of LLMs:

- Expensive to run
 - Slower than simpler methods
 - Require more infrastructure
 - Less predictable
-

Slide 8: When LLMs Are the Right Choice

Decision Criteria for Using LLMs:

Use LLMs when your problem has one or more of these characteristics:

1. Task Involves Complex Language Understanding

What this means:

- Deep comprehension of context is required
- Multiple steps of reasoning needed
- Understanding nuance, sarcasm, or implicit meaning
- Cross-referencing information

Example: "What are the main risks mentioned in this financial report, and how do they compare to last year's risks?"

Why LLM is right:

- Requires understanding financial terminology
- Must identify and categorize risks
- Needs to compare across documents
- Requires synthesis and reasoning

2. Rules Are Impossible to Define

What this means:

- The task is too complex for explicit programming
- Edge cases are infinite
- Human judgment is normally required

Example: "Analyze this job candidate's resume and cover letter to assess cultural fit"

Why LLM is right:

- "Cultural fit" can't be reduced to rules
- Requires understanding subtle communication style
- Needs contextual interpretation
- Human-like judgment needed

3. Training Data is Limited

What this means:

- You don't have thousands of labeled examples
- LLMs come "pre-trained" on vast knowledge
- Can work with few examples (few-shot learning)

Example: "We need to classify medical research papers by methodology, but we only have 20 examples"

Why LLM is right:

- Pre-trained on medical literature
- Can learn from just a few examples
- Classical ML would need thousands of labeled papers

4. Flexibility is Required

What this means:

- Requirements change frequently
- Need to handle new types of requests
- Must adapt without retraining

Example: A customer service bot that must handle:

- Product questions
- Billing issues
- Technical support
- Returns
- And new categories added monthly

Why LLM is right:

- One model handles all categories
- Easy to add new capabilities with prompts
- No retraining needed for new scenarios

Real-World Examples (Expanded):

Example 1: Conversational Assistants

- **Scenario:** A virtual assistant for a bank
- **Challenges:**

- Customers ask unpredictable questions
- Must understand financial terminology
- Needs to maintain context across conversation
- Must handle complex queries like "What's the difference between your two savings accounts, and which saves more tax?"
- **Why LLM:** No other approach can handle this flexibility and complexity

Example 2: Legal Document Analysis

- **Scenario:** Reviewing contracts for specific clauses
- **Challenges:**
 - Legal language is complex and nuanced
 - Same concept expressed in many different ways
 - Requires understanding of legal implications
 - Need to reason across document sections
- **Why LLM:** Can understand legal context and reasoning

Example 3: Customer Support Bots

- **Scenario:** First-line support for software company
 - **Challenges:**
 - Thousands of possible issues
 - Customers describe problems in their own words
 - Need to understand technical concepts
 - Must provide helpful, contextual responses
 - **Why LLM:** Can understand diverse problem descriptions and generate appropriate solutions
-

Slide 9: When LLMs Are the Wrong Choice

Critical Warning:

LLMs are powerful but NOT always the answer. Using them unnecessarily wastes money and creates problems.

Four Situations to AVOID LLMs:

1. Problem is Simple Classification

What this means:

- Clear yes/no or category decision
- Well-defined features
- Straightforward logic

Bad Example: Using GPT-4 for fraud detection

Transaction: \$50 at local grocery store
Use LLM to decide: Fraud or Not Fraud? ❌

Why this is wrong:

- Simple binary classification
- Structured data (amount, location, time, past behavior)
- Classical ML (Logistic Regression, XGBoost) works perfectly
- LLM adds no value but adds huge cost

Right Approach: Use Classical ML

- Features: transaction amount, location, time, user history
- Model: XGBoost or Random Forest
- Cost: Fractions of a penny per prediction
- Speed: Milliseconds
- LLM cost: Several cents per prediction
- LLM speed: Seconds

Cost Comparison:

- Classical ML: Process 1 million transactions for \$10
- LLM: Process 1 million transactions for \$10,000+

2. Structured Data is Dominant

What this means:

- Data is in tables/databases
- Numerical or categorical features
- No significant text to understand

Bad Example: Predicting house prices with LLM

Input: 3 bedrooms, 2000 sq ft, built 1995, suburban location

Using LLM to predict price: ❌

Why this is wrong:

- No complex language to understand
- Just numbers and categories
- Classical regression models are designed for this
- LLM can't add value to numerical relationships

Right Approach: Linear Regression or Tree-based models

- Direct input-output relationship
- Fast and interpretable
- Proven accuracy for structured data

3. Cost or Latency is Critical

What this means:

- Need instant responses (milliseconds)
- Processing millions of requests
- Budget constraints exist

Bad Example: Real-time product recommendations on e-commerce site

User visits product page

Use LLM to generate recommendations: ❌

Why this is wrong:

- Page must load in < 100 milliseconds
- Millions of page views daily
- LLM takes 1-3 seconds per request
- Cost would be astronomical

Latency Comparison:

- Classical ML: 1-10 milliseconds
- LLM: 1000-3000 milliseconds (100-300x slower!)

Cost at Scale: If your app has 1 million requests/day:

- Classical ML: \$50/month
- LLM: \$5,000-\$50,000/month

4. Deterministic Behavior is Required

What this means:

- Need exactly the same output every time
- Regulatory requirements for consistency
- Must be able to audit decisions
- Cannot have variability

Bad Example: Medical diagnosis support

Same patient symptoms
LLM might give different advice each time: ❌

Why this is wrong:

- LLMs can generate different outputs for same input
- Medical decisions must be consistent and auditable
- Regulatory compliance requires deterministic systems
- Need to explain exact decision process

Right Approach: Rule-based system or Classical ML

- Same input → Same output (always)
- Clear audit trail
- Explainable decisions
- Meets regulatory requirements

The Key Lesson:

Just because you CAN use an LLM doesn't mean you SHOULD.

Choose the simplest approach that solves the problem effectively.

Slide 10: Decision Framework Summary

The Four-Question Decision Tree

Use this framework **IN ORDER** for every AI problem:

Question 1: Can Rules Solve This?

Ask yourself:

- Is the logic simple and clear?
- Can I write it in if-then statements?
- Does it rarely change?

If YES → Use Rules

- Cheapest option
- Fastest option

- Easiest to maintain
- Most transparent

Examples:

- Email validation
- Age verification
- Business hours check
- Discount calculation

If NO → Move to Question 2

Question 2: Do I Have Structured Data + Labels?

Ask yourself:

- Is my data in rows and columns?
- Do I have clear features (numbers, categories)?
- Do I have labeled examples of what I want to predict?
- Is this a classification or regression problem?

If YES → Use Classical ML

- Fast and efficient
- Works with moderate data
- Interpretable results
- Low cost

Examples:

- Loan approval
- Stock price prediction
- Customer churn
- Fraud detection

If NO → Move to Question 3

Question 3: Is It Text-Heavy But Limited Scope?

Ask yourself:

- Is the problem mainly about text?
- Is it a well-defined text task?
- Do I need good performance but not perfect understanding?
- Is speed/cost important?

If YES → Use Traditional NLP

- Faster than LLMs
- Much cheaper than LLMs
- Sufficient for focused tasks
- Proven reliability

Examples:

- Spam detection
- Sentiment analysis (simple)
- Keyword extraction
- Document categorization

If NO → Move to Question 4

Question 4: Is It an Open-Ended Language Task?

Ask yourself:

- Does it require deep language understanding?
- Is flexibility crucial?
- Are rules impossible to define?
- Is handling edge cases important?

If YES → Use LLMs

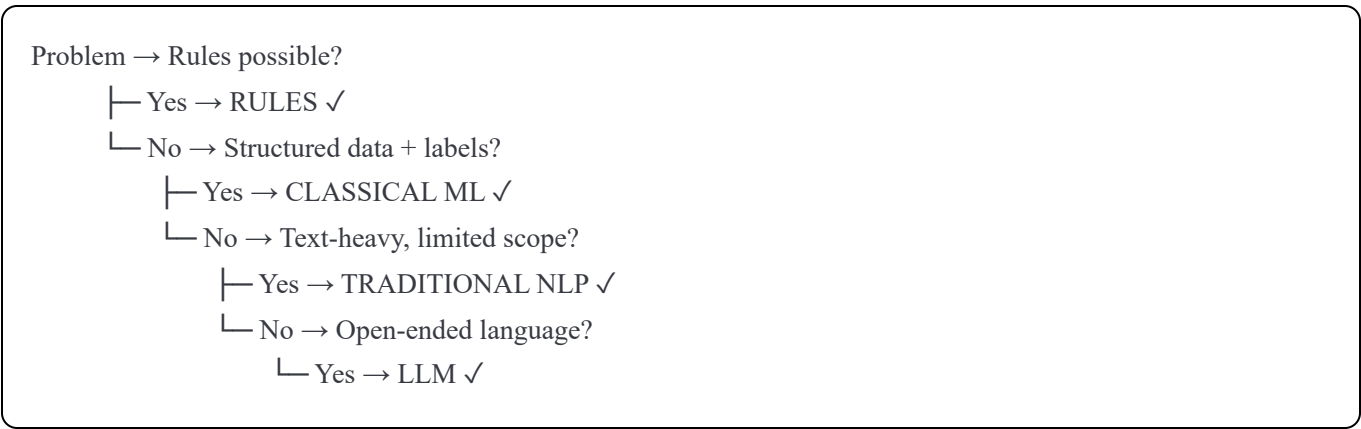
- Most powerful option
- Handles complexity
- Flexible and adaptable
- Best language understanding

Examples:

- Conversational AI
- Complex document analysis
- Creative content generation
- Multi-step reasoning

Accept the cost and latency trade-offs!

Visual Decision Flow:



Key Principle:

Start simple, add complexity only when necessary.

Prefer: Rules → ML → NLP → LLM (in that order)

Slide 11: Interview-Style Example

The Problem:

"We want to detect fraudulent transactions"

This is a common interview question. Let's see how to approach it correctly.

✗ Wrong Approach (What Not to Do):

Candidate says: "We should use GPT-4 to analyze transaction descriptions and detect fraud patterns using natural language understanding."

Why this is wrong:

- Overthinking the problem
- Choosing expensive, complex solution
- Ignoring that fraud is about data patterns, not language

✓ **Correct Approach (Proper Framing):**

Step 1: Analyze the problem structure

"Let's break down what we have:

- **Data type:** Structured transaction data (amounts, locations, times, merchant info, user history)
- **Target variable:** Fraud (yes) or Legitimate (no) — binary outcome
- **Problem type:** Supervised classification
- **Data availability:** We have historical transactions with fraud labels
- **Requirements:** Fast decisions (real-time transaction approval)

Step 2: Apply decision framework

"Following our framework:

1. **Rules?** Too complex — fraud patterns constantly evolve
2. **Classical ML?** Perfect fit! We have:
 - Structured data ✓
 - Clear binary label ✓
 - Fast prediction needed ✓
 - Explainability important (explain to customers) ✓

Step 3: Recommend solution

"I would recommend **Classical ML** — specifically XGBoost or Random Forest because:

Features we can use:

- Transaction amount (is it unusual?)
- Location (different from user's normal location?)
- Time (unusual hour?)
- Merchant category
- User's transaction history
- Velocity (how many transactions in short time?)

Model choice: XGBoost

- Handles complex patterns
- Fast predictions (<10ms)
- Can explain important features
- Proven in fraud detection

Not using LLM because:

- Adds unnecessary cost (hundreds of times more expensive)
- Slower (2-3 seconds vs milliseconds)
- Doesn't improve accuracy for this problem
- Harder to explain decisions"

Interview Success Factors:

What the interviewer looks for:

1. **Structured thinking:** You follow a framework
2. **Understanding constraints:** You consider speed, cost, explainability
3. **Justification:** You explain why NOT using complex solutions
4. **Practicality:** You focus on what works, not what's trendy

Key Phrase to Remember: "LLM is not required for this problem. The data is structured, the outcome is binary, and we can frame this as supervised classification — a perfect fit for Classical ML."

General Interview Strategy:

Always address:

1. What is the data? (structured vs unstructured)
 2. What is the output? (categories, numbers, text)
 3. What is the problem type? (classification, regression, generation)
 4. What are the constraints? (speed, cost, accuracy requirements)
 5. Why NOT the other approaches?
-

Slide 12: Architecture Patterns for NLP & LLMs

Common Architectural Patterns:

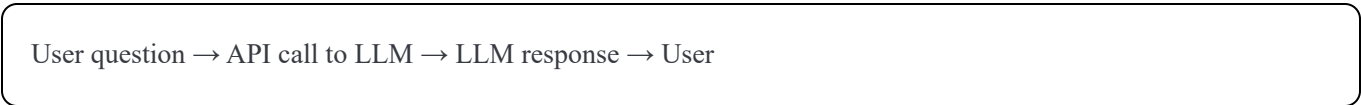
When you decide to use NLP or LLMs, you need to choose **how** to implement them. Here are the three main patterns:

Pattern 1: Prompt-Only API Usage

What it is:

- Send queries directly to an LLM API (like OpenAI GPT-4, Claude)
- No additional infrastructure
- Pure prompt engineering

How it works:



Example:

```
python
response = openai.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "Summarize this document: ..."}]
)
```

When to use:

- Quick prototypes
- Low volume usage (<1000 queries/day)
- Generic tasks (summarization, translation)
- Don't have specific domain knowledge needs

Advantages:

- ✓ Fastest to implement (hours)
- ✓ No infrastructure needed
- ✓ Always up-to-date model

Disadvantages:

- ✗ Expensive at scale
- ✗ No domain-specific knowledge
- ✗ Internet dependency
- ✗ Limited customization

Cost Example:

- 1000 queries/day × \$0.02/query = \$600/month

Pattern 2: Retrieval-Augmented Generation (RAG)

What it is:

- Combine LLM with your own knowledge base
- Retrieve relevant documents first, then use LLM to answer
- Gives LLM context it wasn't trained on

How it works:

User question

→ Search your documents

→ Find relevant passages

→ Send passages + question to LLM

→ LLM generates answer based on your documents

Example Use Case: Company documentation chatbot

1. User asks: "What's our return policy for international orders?"
2. System searches company knowledge base
3. Finds relevant policy documents
4. Sends documents + question to LLM
5. LLM answers based on actual company policy

Technical Components:

- **Vector Database:** Stores document embeddings (Pinecone, Weaviate, Chroma)
- **Embedding Model:** Converts text to searchable vectors
- **LLM:** Generates answers from retrieved context

When to use:

- Need domain-specific knowledge
- Have internal documents/data
- Want accurate, grounded answers
- Medium to high volume usage

Advantages:

- ✓ Incorporates your specific data
- ✓ More accurate than pure LLM
- ✓ Reduces hallucinations
- ✓ Can cite sources
- ✓ Balances cost and capability

Disadvantages:

- ✗ More complex to build
- ✗ Requires vector database
- ✗ Need to maintain knowledge base
- ✗ Still has API costs

Cost Example:

- Vector DB: \$50-200/month

- LLM API: \$300-1000/month (more efficient than pure prompting)
- Total: \$350-1200/month

Reference: Pinecone RAG Guide (<https://www.pinecone.io/learn/retrieval-augmented-generation/>)

Pattern 3: Fine-Tuned Models

What it is:

- Take a base LLM
- Train it further on your specific data
- Creates a customized model for your use case

How it works:

Base model (GPT, Llama)

→ + Your training data

→ Fine-tuning process

→ Custom model specialized for your task

Example: Customer service for a tech company

Training data: 10,000 historical customer conversations **Result:** Model learns company-specific:

- Product names and features
- Common issues and solutions
- Company tone and style

When to use:

- High volume usage (>100,000 queries/month)
- Very specific domain/style needed
- Latency is critical
- Cost savings justify upfront investment

Advantages:

- ✓ Best performance for specific task
- ✓ Cheaper per query at scale
- ✓ Can self-host (no API dependency)
- ✓ Lower latency
- ✓ More control

Disadvantages:

- ✗ High upfront cost (\$5,000-\$50,000)
- ✗ Requires ML expertise
- ✗ Need significant training data
- ✗ Ongoing maintenance
- ✗ Slow to update

Cost Example at Scale:

- Initial fine-tuning: \$10,000
- Self-hosting: \$500/month
- Cost per 1M queries: \$100
- (vs API: \$10,000 for 1M queries)

The Golden Rule:

"Prefer simpler architectures unless complexity is justified."

Decision order:

1. Try prompt-only first (simplest)
2. Add RAG if you need specific knowledge
3. Consider fine-tuning only at high scale

Don't over-engineer! Many production systems run successfully on simple prompt-only or RAG architectures.

PART 2: COST, LATENCY & ARCHITECTURE TRADE-OFFS (Day 9 Content)

Slide 13: Why Cost & Latency Matter

The Reality Check:

A model can be:

- ✓ **Accurate** — 99% performance on test set
- ✓ **Elegant** — Beautiful code, state-of-the-art techniques

And STILL BE:

- ✗ **Too slow** — Takes 5 seconds when users need instant results
- ✗ **Too expensive** — Costs \$10,000/month when budget is \$100/month

Production Success Requires Balance:

You need **all four qualities**:

1. **Accuracy:** Good enough performance
2. **Speed (Latency):** Fast enough for users
3. **Cost:** Within budget
4. **Reliability:** Consistent and stable

Missing ANY of these means failure in production.

Real-World Example:

Scenario: E-commerce product recommendation system

Version 1 (The "Perfect" Model):

- Accuracy: 95% (amazing!)
- Latency: 3 seconds per recommendation
- Cost: \$20,000/month
- **Result:** ✗ FAILED
 - Users left website before recommendations loaded
 - Company couldn't afford the cost

Version 2 (The Production Model):

- Accuracy: 88% (good enough!)
- Latency: 50 milliseconds
- Cost: \$800/month
- **Result:** ✓ SUCCESS

- Users happy with speed
- Budget sustainable
- Business value delivered

Key Lesson:

"Perfect is the enemy of good."

The best model is the one that:

- Meets minimum accuracy requirements
- Delivers results fast enough
- Stays within budget
- Actually ships to production

Remember: A model in production at 85% accuracy creates more value than a model at 95% accuracy that never deploys due to cost or speed issues.

Slide 14: Cost Drivers in AI Systems

Understanding the Full Cost of AI:

Many people only think about "training cost," but that's just one piece. Here are ALL the major cost components:

1. Training Cost

What it includes:

- Compute resources (GPUs/TPUs)
- Time spent training
- Experimentation and iteration
- Data processing

Examples:

- Classical ML: \$10-100 to train
- Deep Learning: \$100-1,000 to train
- Large Language Model: \$1M-10M to train from scratch (rare)
- Fine-tuning LLM: \$1,000-50,000

Key Point: This is a ONE-TIME cost (per model version)

2. Inference Cost (Most Important!)

What it is: The cost of making predictions after the model is trained

Why it matters:

- This is RECURRING — every single prediction costs money
- At scale, this dominates all other costs
- Often overlooked during development

Cost Comparison per 1,000 predictions:

- Simple ML model (self-hosted): \$0.01
- Complex ML model (self-hosted): \$0.10
- LLM via API: \$10-50 (1000-5000x more expensive!)

Real Example: If your app gets 1 million requests per month:

- Classical ML: \$10-100/month

- LLM API: \$10,000-50,000/month

The Scary Math:

Company launches chatbot

Week 1: 1,000 users → \$500 in LLM costs ✓

Month 2: 10,000 users → \$5,000 ✓

Month 6: 100,000 users → \$50,000 ⚠️

Year 2: 1M users → \$500,000 ❌ UNSUSTAINABLE

3. Data Storage Cost

What it includes:

- Training data storage
- Model storage
- Prediction logs
- Feature stores

Examples:

- 1TB data on cloud: \$20-50/month
- Growing at 10TB/month: adds \$200-500/month
- Real-time features: \$100-1,000/month

Grows over time as:

- You collect more training data
- Store more predictions for monitoring
- Keep historical data for retraining

4. Monitoring and Retraining Cost

What it includes:

- Performance monitoring systems
- Data drift detection
- Model retraining
- A/B testing infrastructure
- On-call engineer time

Examples:

- Monitoring tools: \$50-500/month
- Retraining weekly: \$100-1,000/month
- Engineer maintenance: \$5,000-15,000/month (salary portion)

The Hidden Multiplier Effect:

LLMs Add Recurring Inference Costs that scale with usage:

Traditional ML one-time costs → Predictable budget
LLM continuous API costs → Unpredictable, can explode

Budget Planning:

Classical ML Project:

- Training: \$1,000 (one-time)
- Infrastructure: \$200/month
- Total Year 1: \$3,400

LLM Project:

- Fine-tuning: \$10,000 (one-time)
- API costs: Start \$1,000/month, grow to \$10,000/month
- Total Year 1: \$66,000+ (and growing!)

Critical Question to Ask:

"What happens when usage 10x?"

- Classical ML: Cost stays roughly the same
- LLM API: Cost 10x (potentially unsustainable)



Slide 15: Latency Considerations

What is Latency?

Latency: The time between sending a request and receiving a response.

In AI systems: Time from input → prediction → output

Why Latency Matters:

1. User Experience

Human perception thresholds:

- **<100ms:** Feels instant (excellent UX)
- **100-300ms:** Slight delay, still good
- **300-1000ms:** Noticeable delay (users getting impatient)
- **>1000ms (1 second):** Frustrating (users may abandon)
- **>3 seconds:** Most users will leave

Real-World Impact:

- **Amazon found:** Every 100ms delay → 1% drop in sales
- **Google found:** 500ms delay → 20% drop in traffic

2. System Scalability

The throughput problem:

If each request takes 3 seconds:

- **Single server** can handle: 1 request / 3 seconds = 0.33 requests/second = 20 requests/minute
- To handle 1000 requests/minute: Need 50 servers!
- Cost: 50 servers × \$100/month = \$5,000/month

If each request takes 100ms:

- **Single server** can handle: 10 requests/second = 600 requests/minute
- To handle 1000 requests/minute: Need 2 servers
- Cost: 2 servers × \$100/month = \$200/month

Latency directly impacts cost!

Typical Latency by Model Type:

Classical ML Models:

- Linear/Logistic Regression: 1-5ms
- Random Forest: 10-50ms
- XGBoost: 20-100ms
- Neural Networks: 50-200ms

Traditional NLP:

- TF-IDF classification: 5-20ms
- Word embedding similarity: 10-50ms

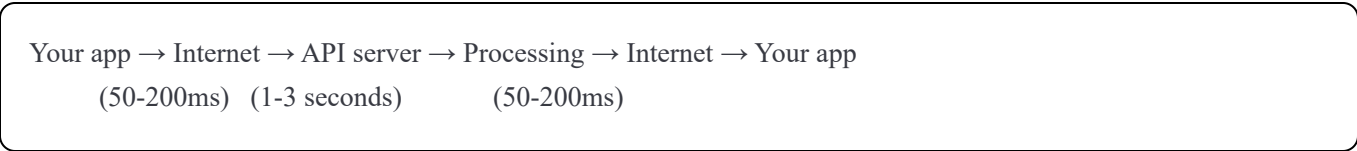
Large Language Models:

- Simple query: 1000-3000ms (1-3 seconds)
- Complex query: 3000-10,000ms (3-10 seconds)
- With RAG: add 100-500ms for retrieval

LLMs are typically 10-100x slower than traditional ML models.

Network Dependency Issue:

LLM APIs have added latency:



Total: 1.1-3.4 seconds minimum

Self-hosted models:



Total: 1-3 seconds (saves network latency)

Strategies to Manage Latency:

1. Caching

What it is: Store results of common queries

Example:



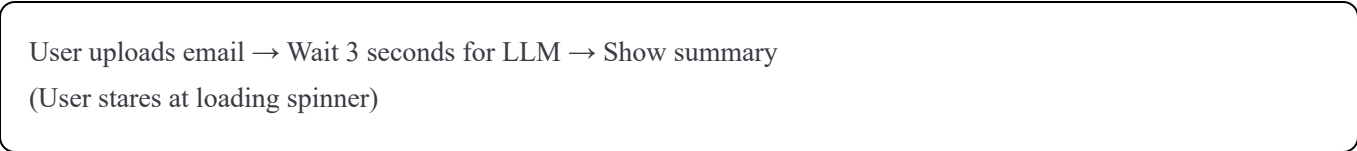
Impact:

- 99%+ of queries served in milliseconds
- Massive cost savings (don't pay for repeated API calls)
- Only fresh/unique queries hit the slow path

2. Async Processing

What it is: Don't make users wait for slow operations

Example: Email summarization ❌ Bad (Synchronous):



✓ Good (Asynchronous):

User uploads email → Immediate confirmation "Processing..."
→ LLM processes in background
→ User notified when ready (or checks back later)
(User can continue other work)

Use cases:

- Batch processing
- Report generation
- Document analysis
- Content moderation

3. Model Optimization

Techniques:

- Use smaller, faster models when possible
- Quantization (reduce model precision, faster inference)
- Model distillation (train smaller model to mimic larger one)

4. Hybrid Approaches

Fast model first, slow model if needed:

Simple query → Fast model (50ms) → Return immediately
Complex query → Slow model (2 seconds) → Return when ready

Example:

"What's your return policy?" → Fast cached response
"Analyze this contract for risks" → Route to LLM

Key Principle:

"Use the fastest approach that meets accuracy requirements."

Don't default to the most powerful (and slowest) model for every task.

Slide 16: Architecture Trade-offs

Every Architecture Decision Has Trade-offs:

There is **no perfect architecture**. Every choice involves compromises across multiple dimensions:

Three Key Dimensions:

1. Scalability

Definition: Can the system handle growth?

Trade-off:

- **High scalability:** Complex infrastructure, higher upfront cost
- **Low scalability:** Simple setup, but hits limits quickly

Example:

- **Serverless API:** Scales automatically, but expensive per request
- **Fixed servers:** Cheaper at stable load, but can't handle traffic spikes

2. Maintainability

Definition: How easy is it to update, debug, and manage?

Trade-off:

- **High maintainability:** Simpler design, but may sacrifice performance
- **Low maintainability:** Optimized performance, but harder to modify

Example:

- **Monolithic app:** Everything in one place, easy to understand
- **Microservices:** Each service independent, but complex coordination

3. Cost

Definition: Total cost of ownership (development + operation)

Trade-off:

- **Lower operational cost:** Higher development complexity
- **Lower development cost:** Higher operational cost

Example:

- **Cloud API:** Fast to build, expensive to run
- **Self-hosted:** Expensive to build, cheaper to run at scale

Common Architecture Decisions:

Decision 1: Batch vs Real-Time Inference

Batch Processing:

- **What:** Process many predictions together on schedule
- **When:** Once per hour/day/week
- **Pro:** Lower cost, simpler infrastructure
- **Con:** Not instant results

Example: Email spam filtering

- Process all emails at once every hour
- Users see results within an hour
- Cost: 10x cheaper than real-time

Real-Time Processing:

- **What:** Process each request immediately
- **When:** Instantly on demand
- **Pro:** Instant results, better UX
- **Con:** Higher cost, more complex infrastructure

Example: Credit card fraud detection

- Must approve/decline transaction in <200ms
- Can't wait for batch processing
- Cost: Higher, but necessary

Choose based on:

- Do users need instant results?
- Is delay acceptable?
- What's the cost difference?

Decision 2: Cloud API vs Self-Hosted Model

Cloud API (e.g., OpenAI GPT-4):

Pros:

- ✓ Fast to implement (hours/days)
- ✓ No infrastructure management
- ✓ Always up-to-date
- ✓ Pay per use (good for low volume)

Cons:

- ✗ Expensive at scale
- ✗ Network dependency
- ✗ Less control
- ✗ Data privacy concerns (sending data to third party)

When to use:

- Prototyping
- Low volume (<1000 requests/day)
- Don't have ML infrastructure
- Speed to market is critical

Self-Hosted Model:

Pros:

- ✓ Lower cost at scale
- ✓ Full control
- ✓ Data privacy (everything on your servers)
- ✓ Customizable

Cons:

- ✗ High upfront investment
- ✗ Requires infrastructure
- ✗ Need ML expertise
- ✗ Maintenance burden

When to use:

- High volume (>100,000 requests/day)
- Data sensitivity (healthcare, finance)
- Have ML team
- Long-term project

Cost Crossover Example:

Volume: 10,000 requests/day

Cloud API:

- \$0.02 per request
- $10,000 \times \$0.02 \times 30 \text{ days} = \$6,000/\text{month}$

Self-hosted:

- Server: \$500/month
- ML engineer (partial time): \$3,000/month
- Total: \$3,500/month

→ Self-hosted cheaper above ~5,000 requests/day

Decision 3: Simple Model vs Complex Model

Simple Model (e.g., Logistic Regression):

- **Pro:** Fast, cheap, interpretable
- **Con:** Lower accuracy potential

Complex Model (e.g., Deep Learning):

- **Pro:** Higher accuracy potential
- **Con:** Slower, expensive, black box

The Question: Is the accuracy improvement worth the cost?

Example:

- Simple model: 85% accuracy, 10ms latency, \$100/month
- Complex model: 88% accuracy, 500ms latency, \$5,000/month

Decision depends on:

- Is 3% accuracy improvement valuable?
- Can users tolerate 500ms latency?
- Is \$4,900 extra cost justified?

The Universal Trade-off Matrix:

Dimension	Simple Arch		Complex Arch	
	-----	-----	-----	
Development	Fast		Slow	
Operational \$	High		Low (scale)	
Latency	Good		Variable	
Scalability	Limited		High	
Maintenance	Easy		Hard	

Key Principle:

"Choose the simplest architecture that meets your requirements."

Start simple, add complexity only when necessary and justified.

Slide 17: Example Architecture - Chatbot

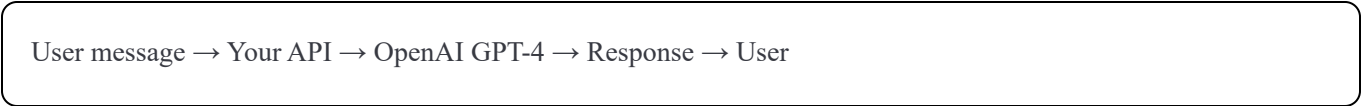
Business Need:

Build a customer service chatbot

Three Architecture Options:

Option 1: LLM API (Fast to Build, Costly)

Architecture:



Implementation:

- Direct calls to GPT-4 API
- Simple prompt engineering
- Minimal custom code

Pros:

- ✓ Build in 1-2 weeks
- ✓ Excellent conversation quality
- ✓ Handles complex queries
- ✓ No ML expertise needed

Cons:

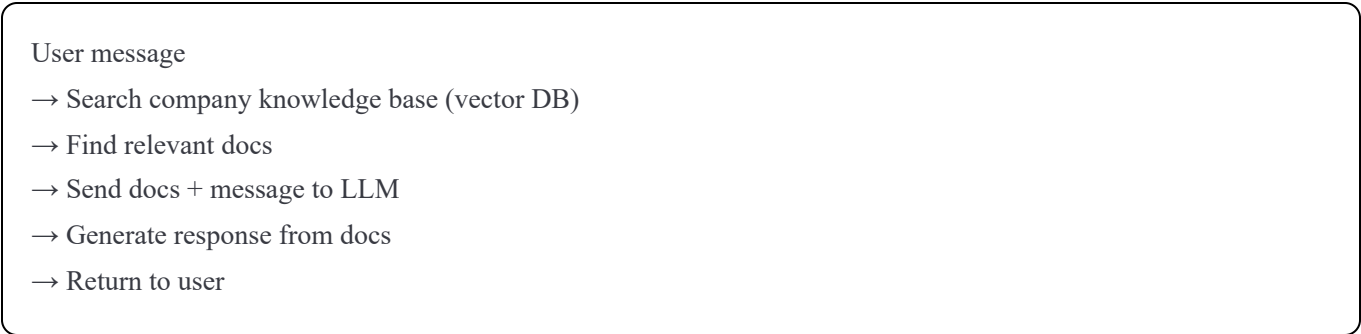
- ✗ \$0.03 per conversation
- ✗ At 10,000 conversations/day: \$9,000/month
- ✗ Costs grow with usage
- ✗ Dependent on external API

Good for:

- MVP/prototype
- Low volume (< 1,000 conversations/day)
- Need to launch fast
- Budget > \$5,000/month

Option 2: RAG-Based Chatbot (Balanced)

Architecture:



Components:

- Vector database (Pinecone/Chroma): \$100/month
- Embedding API: \$0.001 per query
- LLM API (GPT-3.5): \$0.01 per response
- Custom orchestration code

Implementation complexity: 4-6 weeks

Pros:

- ✓ Incorporates company-specific knowledge
- ✓ More accurate than pure LLM
- ✓ Reduces hallucinations
- ✓ Can cite sources
- ✓ Better cost at medium scale

Cons:

- ✗ More complex to build
- ✗ Requires maintaining knowledge base
- ✗ Still has API costs

Cost at scale:

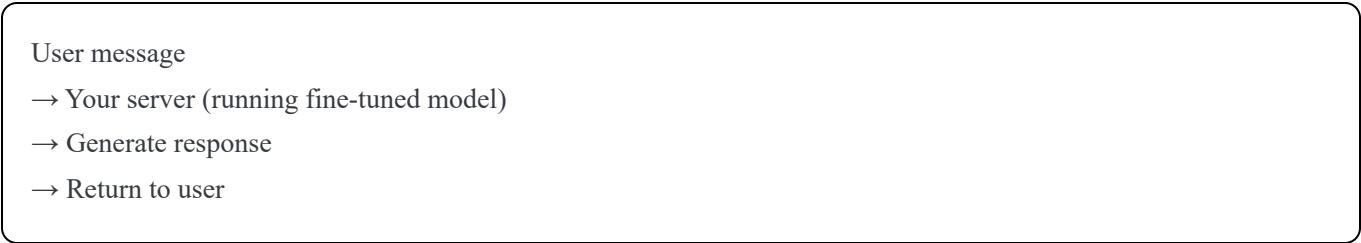
- 10,000 conversations/day
- Vector DB: \$100/month
- Embeddings: $10k \times \$0.001 \times 30 = \$300/\text{month}$
- LLM calls: $10k \times \$0.01 \times 30 = \$3,000/\text{month}$
- **Total: \$3,400/month** (vs \$9,000 for pure LLM)

Good for:

- Production systems
- Need company-specific answers
- Medium-high volume (1,000-50,000/day)
- Balance of cost and quality

Option 3: Fine-Tuned Local Model (Complex, Cheaper at Scale)

Architecture:



Implementation:

1. Collect 10,000+ conversation examples
2. Fine-tune base model (Llama, Mistral)
3. Deploy on your infrastructure
4. Optimize for speed

Implementation complexity: 3-6 months

Pros:

- ✓ No per-query costs
- ✓ Full control
- ✓ Data privacy
- ✓ Lowest cost at very high scale
- ✓ Customized for your use case

Cons:

- ✗ High upfront cost (\$20k-50k)
- ✗ Requires ML expertise
- ✗ Long development time
- ✗ Ongoing maintenance
- ✗ Need significant training data

Cost at scale:

- Infrastructure: \$1,000-2,000/month
- ML engineer maintenance: \$5,000/month (partial)
- **Total: ~\$6,000-7,000/month FIXED** (doesn't grow with usage)

Good for:

- Very high volume (>50,000 conversations/day)
- Have ML team
- Long-term commitment
- Data sensitivity requirements

Decision Matrix by Volume:

Daily Volume	Best Choice	Monthly Cost
-----	-----	-----
< 1,000	LLM API	\$1,000
1k - 10k	RAG	\$3,000
10k - 50k	RAG	\$15,000
> 50k	Fine-tuned	\$7,000 (fixed)

The Crossover Point:

At ~30,000 conversations/day:

- LLM API: \$27,000/month (and growing)
- RAG: \$10,000/month (and growing)
- Fine-tuned: \$7,000/month (stays fixed)

Fine-tuning becomes cost-effective at high scale.

Key Decision Factors:

1. **Current volume:** Where are you now?
2. **Expected growth:** Where will you be in 6 months?
3. **Budget:** What can you afford?
4. **Expertise:** Do you have ML team?
5. **Time to market:** How fast must you launch?

Recommended Approach:

Phase 1 (MVP): Start with LLM API

- Launch fast, validate concept
- Learn what users need
- Low upfront investment

Phase 2 (Production): Migrate to RAG

- Add company knowledge
- Reduce costs as volume grows
- Better quality

Phase 3 (Scale): Consider fine-tuning

- Only if volume justifies it
 - When costs become unsustainable
 - If you have ML expertise
-

Slide 18: Example Architecture - Prediction System

Business Need:

Predict customer churn (will they cancel subscription?)

Why This is NOT an LLM Problem:

Data structure:

Customer ID	Age	Months Active	Support Tickets	Last Login	Churn	
-----	----	-----	-----	-----	-----	
12345	28	14	2	5 days ago	No	
67890	45	3	8	30 days ago	Yes	

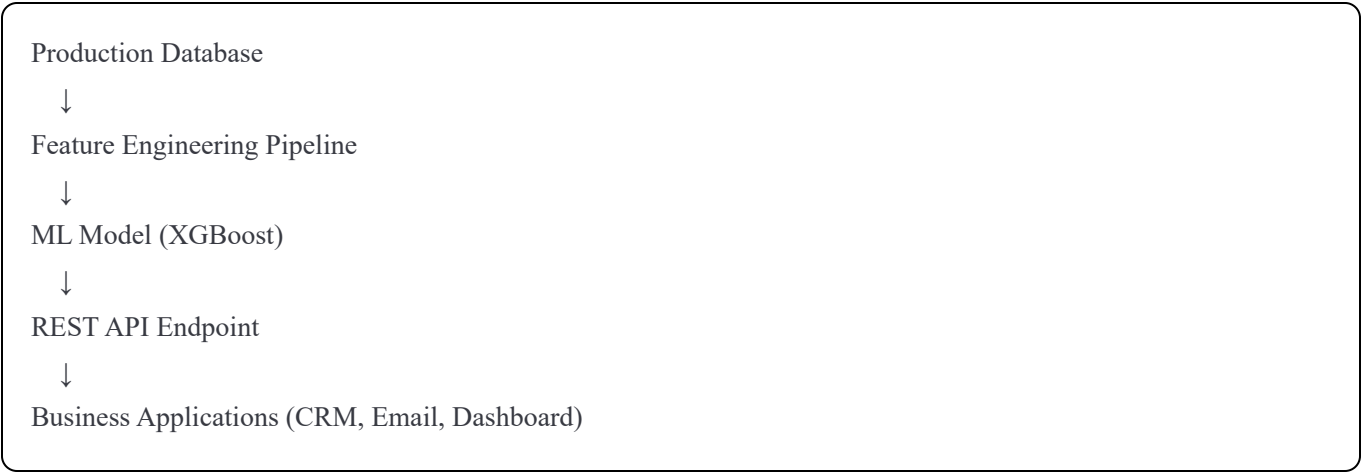
Key observations:

- Data is structured (rows and columns)
- Clear binary target (Churn: Yes/No)
- Numerical and categorical features
- No complex language to understand
- Historical labeled data available

Conclusion: This is a **Classical ML** problem, NOT an LLM use case.

The Right Architecture: Classical ML as Microservice

Architecture Diagram:



Components:

1. Feature Engineering Pipeline

- Aggregate user data
- Calculate features (e.g., days since last login, support tickets per month)
- Handle missing values
- Transform categorical variables

2. ML Model

- **Algorithm:** XGBoost (handles complex patterns, fast, reliable)
- **Training:** Retrain weekly with new data
- **Features:** 20-30 predictive features
- **Output:** Churn probability (0-100%)

3. Deployed as Microservice

- **API Endpoint:** POST /predict/churn
- **Input:** Customer ID or feature vector
- **Output:** {"churn_probability": 0.73, "risk_level": "high"}
- **Latency:** 10-20 milliseconds
- **Hosting:** Docker container, Kubernetes for scaling

Why This Architecture is Better:

1. Low Latency

Classical ML Model:

- Prediction time: 10-20 milliseconds
- Can process 50-100 predictions per second per server
- User gets instant results

If we used LLM (wrong choice):

- Prediction time: 1-3 seconds (100-300x slower!)
- Can process ~1 prediction per second
- User waits, poor experience
- Need 100x more servers

2. Predictable Cost

Classical ML Costs:

- **Training:** \$50/week (retraining)
- **Hosting:** \$200/month (server)
- **Storage:** \$20/month
- **Total:** ~\$400/month FIXED
- **Cost per prediction:** \$0.0001

If we used LLM:

- **API costs:** \$0.02 per prediction
- For 1M predictions/month: \$20,000
- **50x more expensive!**

Scaling comparison:

Volume/Month	Classical ML	LLM API	Savings
-----	-----	-----	-----
100k	\$400	\$2,000	\$1,600
1M	\$400	\$20,000	\$19,600
10M	\$800	\$200,000	\$199,200

3. Easier Monitoring

What to monitor in Classical ML:

- Feature distributions (are they changing?)
- Prediction distribution (% predicted to churn)
- Model accuracy (track actual vs predicted)
- API latency
- Error rates

Simple dashboards:

- Line charts of daily predictions
- Confusion matrix (true/false positives)
- Feature importance tracking

Monitoring tools:

- Prometheus + Grafana (open source)

- AWS CloudWatch
- Datadog

LLM monitoring challenges:

- Harder to understand why predictions change
- Black box behavior
- Inconsistent outputs
- Difficult to debug

4. Explainability

Business needs to understand WHY customer might churn:

XGBoost provides:

Customer ID: 67890

Churn Risk: 73% (High)

Top factors:

1. Days since last login: 30 (high risk)
2. Support tickets: 8 in 3 months (high risk)
3. Subscription length: 3 months (new customer, higher risk)
4. Usage declined: 80% drop in last month

Value:

- Customer service can take action: "Call customer, offer support"
- Business understands patterns: "New customers with support issues need attention"
- Regulatory compliance: Can explain automated decisions
- Builds trust with stakeholders

LLMs are black boxes:

- Hard to explain specific predictions
- Less transparency
- Regulatory challenges

Implementation Details:

Feature Engineering Example:

```
python

# Calculate risk features
days_since_login = (today - last_login_date).days
support_ticket_rate = total_tickets / months_active
usage_trend = (recent_usage - previous_usage) / previous_usage

# Binary features
has_payment_issues = payment_failures > 0
is_new_customer = months_active < 6
```

Model Training:

```
python
```



```
from xgboost import XGBClassifier

model = XGBClassifier(
    max_depth=6,
    learning_rate=0.1,
    n_estimators=100
)

model.fit(X_train, y_train)
```

API Deployment:

```
python

from flask import Flask, request

@app.route('/predict/churn', methods=['POST'])
def predict():
    customer_id = request.json['customer_id']
    features = get_customer_features(customer_id)
    prediction = model.predict_proba(features)[0][1]

    return {
        'customer_id': customer_id,
        'churn_probability': prediction,
        'risk_level': 'high' if prediction > 0.6 else 'medium' if prediction > 0.3 else 'low'
    }
```

Key Takeaway:

For structured data prediction problems:

- ✓ Use Classical ML
- ✓ Deploy as microservice
- ✓ Fast, cheap, explainable
- ✗ Don't use LLM just because it's trendy

The right tool for the right job.

Slide 19: Trade-off Matrix

The Reality: No Perfect Solution

Every AI approach has strengths and weaknesses. **No solution wins on all axes.** You must choose which trade-offs you can accept.

The Four Key Dimensions:

1. Accuracy

How good are the predictions/outputs?

2. Cost

How much does it cost to build and run?

3. Latency

How fast does it respond?

4. Complexity

How hard is it to build and maintain?

Comparison Matrix:

Approach	Accuracy	Cost	Latency	Complexity
----------	----------	------	---------	------------

Rules	Low	★★★★★	★★★★★	★★★★★		
Classical ML	Medium	★★★★	★★★★★	★★★		
Traditional NLP	Medium	★★★★	★★★★	★★★		
LLM (API)	High	★	★★	★★★★		
LLM (Fine-tuned)	Highest	★★	★★★	★		
★★★★★ = Best (Lowest cost, fastest, simplest)						
★ = Worst (Highest cost, slowest, most complex)						

Detailed Analysis:

Rules-Based Systems

Strengths:

- Cost: ★★★★★ Almost free to run
- Latency: ★★★★★ Microseconds
- Complexity: ★★★★★ Easy to build and understand

Weaknesses:

- Accuracy: Limited — only works for simple, well-defined problems

Example: Password validation

- Perfect for this use case
- Would fail for complex decisions

Classical ML

Strengths:

- Latency: ★★★★★ Very fast (milliseconds)
- Cost: ★★★★★ Cheap to train and run
- Accuracy: Medium-High for structured data

Weaknesses:

- Complexity: ★★★ Requires data science skills
- Accuracy: Struggles with unstructured data or complex language

Example: Credit scoring

- Excellent accuracy on structured financial data
- Can't understand loan application essays

Traditional NLP

Strengths:

- Latency: ★★★★★ Fast (10-50ms)
- Cost: ★★★★★ Moderate training, cheap inference
- Accuracy: Good for focused text tasks

Weaknesses:

- Complexity: ★★★ Requires NLP expertise
- Accuracy: Limited compared to LLMs on complex language

Example: Spam filtering

- Excellent for this specific task
- Can't handle open-ended conversation

LLM via API

Strengths:

- **Accuracy:** ★★★★★ Excellent language understanding
- **Complexity:** ★★★★★ Easy to implement (API calls)
- **Flexibility:** Can handle many tasks

Weaknesses:

- **Cost:** ★ Very expensive per query
- **Latency:** ★★ Slow (1-3 seconds)

Example: Customer service chatbot

- Excellent conversation quality
- Expensive at scale
- Need backup plan for slow responses

Fine-Tuned LLM (Self-Hosted)

Strengths:

- **Accuracy:** ★★★★★★ Best for specific domain
- **Cost:** ★★ Better than API at high scale
- **Control:** Complete ownership

Weaknesses:

- **Complexity:** ★ Hardest to build
- **Upfront Cost:** Very high (\$20k-50k)
- **Latency:** ★★★ Still slower than Classical ML

Example: Legal document analysis at scale

- Best accuracy for legal text
- Justified only at high volume
- Requires expert team

Real-World Trade-off Examples:

Example 1: Startup with Limited Budget

Situation:

- Budget: \$500/month
- Volume: 1,000 users
- Timeline: Need to launch in 1 month

Wrong Choice: Fine-tuned LLM

- Costs \$50k upfront
- Takes 6 months to build
- Beyond budget and timeline

Right Choice: LLM API

- Launch in 1 week
- Cost: \$500/month
- Accept higher cost for faster market entry

- Plan to optimize later if successful

Example 2: High-Volume E-commerce

Situation:

- Volume: 10 million predictions/day
- Latency requirement: <100ms
- Budget: Limited

Wrong Choice: LLM API

- Would cost \$200k+/month
- Latency: 2-3 seconds (too slow)
- Unsustainable

Right Choice: Classical ML

- Cost: \$1,000/month
- Latency: 10ms (perfect)
- Proven reliability

Example 3: Healthcare Diagnosis Support

Situation:

- Need high accuracy
- Must explain decisions (regulatory)
- Budget: Moderate
- Volume: Medium

Wrong Choice: LLM API

- Black box decisions
- Can't explain confidently
- Regulatory issues

Right Choice: Ensemble of Classical ML + Rules

- Explainable decisions
- Meet regulatory requirements
- High accuracy through ensemble
- Acceptable cost

The Decision Framework:

Step 1: Rank your priorities (1 = most important, 4 = least)

__ Accuracy

__ Cost

__ Latency

__ Simplicity

Step 2: Eliminate approaches that fail critical requirements

- If latency must be <100ms: Eliminate LLMs
- If budget < \$5k/month: Eliminate LLM APIs at scale
- If must be explainable: Consider Classical ML over LLMs

Step 3: Choose the simplest approach that meets remaining needs

Critical Principle:

"You cannot optimize for everything. Choose your battles."

- Need accuracy AND speed? Pay more money
- Need low cost AND high volume? Accept complexity of self-hosting
- Need simplicity AND flexibility? Pay for LLM API costs

There is no free lunch.

Slide 20: Common Architecture Mistakes

The Four Deadly Mistakes

These mistakes are extremely common and cause projects to fail **after** months of work. Learn to avoid them:

Mistake 1: Choosing LLMs for Everything

The Problem: Using LLMs has become trendy. People default to LLMs without considering if they're appropriate.

Why it's wrong:

- LLMs are expensive overkill for many problems
- Adds unnecessary complexity
- Slower than alternatives
- Harder to debug and maintain

Real Example: Company: E-commerce site **Mistake:** Used GPT-4 to classify products into categories

Data: Product name, description (structured text) **Result:**

- Cost: \$5,000/month
- Latency: 2 seconds per product
- Accuracy: 92%

Better Approach: Classical text classification (TF-IDF + Logistic Regression) **Result:**

- Cost: \$50/month (100x cheaper!)
- Latency: 5 milliseconds (400x faster!)
- Accuracy: 91% (nearly the same!)

The Fix: Ask: "What's the simplest approach that could work?"

- Try rules first
- Then Classical ML
- Only use LLM if others fail

Warning Signs:

- "Let's use GPT for..." (before understanding the problem)
- No consideration of alternatives
- No cost/latency analysis

Mistake 2: Ignoring Cost at Design Time

The Problem: Focusing only on building the model, ignoring operational costs until deployment.

Why it's wrong:

- Costs can be unsustainable in production
- Too late to redesign after building

- Forces emergency architecture changes
- Can kill the entire project

Real Example: Company: Social media platform **Mistake:** Built content moderation with GPT-4 **Development:** 2 months, worked beautifully in testing **Launch:** 1 million posts/day to moderate **Week 1 bill:** \$30,000 in API costs **Week 2:** Emergency meeting, CEO furious **Result:** Entire system had to be rebuilt from scratch with Classical ML

What went wrong:

Planning: "LLM is perfect for understanding context!"
Development: Only tested with 100 posts/day
Launch: Reality of 1M posts/day hits
Math: 1M posts × \$0.03/post × 30 days = \$900,000/month ❌

The Fix: Always calculate costs BEFORE building:

1. **Estimate volume:** How many predictions per day/month?
2. **Calculate cost per prediction:** API pricing or server costs
3. **Project monthly cost:** Volume × Cost per prediction
4. **Compare to budget:** Is this sustainable?
5. **Calculate breakeven:** When does self-hosting become cheaper?

Example calculation:

Expected Volume: 100,000 requests/day = 3M/month

Option A: LLM API
Cost: \$0.02 per request
Monthly: 3M × \$0.02 = \$60,000 ❌ Over budget!

Option B: Classical ML
Server: \$500/month
Training: \$200/month
Total: \$700/month ✓ Within budget!

Warning Signs:

- No cost projection in design doc
- "We'll optimize costs later"
- Not considering scaling
- Only testing with small data volumes

Mistake 3: No Fallback Strategies

The Problem: Assuming the primary system will always work perfectly, with no backup plan.

Why it's wrong:

- APIs fail
- Models become overloaded
- Latency spikes occur
- Errors happen

Without fallback: System crashes, users see errors, business loses money

Real Example: Company: Customer service chatbot **Mistake:** 100% dependent on OpenAI API, no fallback **Incident:** OpenAI API went down for 2 hours **Result:**

- Chatbot completely unavailable
- Customers couldn't get help
- Negative reviews flooded in

- Lost sales estimated at \$50,000

The Fix: Implement graceful degradation:

Strategy 1: Fallback to simpler system

Primary: LLM for complex queries

↓ If fails

Fallback: Rule-based FAQ answers

↓ If fails

Final: "Service temporarily unavailable, contact support@..."

Strategy 2: Queue and retry

Request fails

↓

Add to queue

↓

Retry with exponential backoff

↓

Process when service recovers

Strategy 3: Cache common queries

"What's your return policy?" → Cached answer (instant)

If cache miss → Try API

If API fails → Return "Please try again"

Real-World Fallback Architecture:

User query

↓

Check cache → Hit? → Return cached response (5ms)

↓ Miss

Call LLM API → Success? → Return + cache response

↓ Fail

Try backup LLM provider → Success? → Return response

↓ Fail

Return FAQ matches (keyword search) → "Here are related articles..."

↓ No matches

Graceful error: "Service temporarily unavailable. Email us at..."

Warning Signs:

- Single point of failure
- No error handling strategy
- "The API is reliable enough"
- No testing with failures

Mistake 4: No Monitoring Plan

The Problem: Deploying a model without systems to track its performance and health.

Why it's wrong:

- Models degrade over time (data drift)
- Can't detect when model starts failing
- Can't diagnose issues
- Silent failures harm business

Real Example: Company: Fraud detection system **Mistake:** Deployed model with no monitoring **What happened:**

- Fraudsters changed tactics after 3 months

- Model accuracy dropped from 95% to 70%
- Nobody noticed for 2 months
- Fraud losses increased \$200,000
- Only discovered when customer complained

If monitoring existed:

- Alert when accuracy drops below 90%
- Detect change in fraud patterns
- Retrain model immediately
- Loss prevented: \$180,000

The Fix: Implement comprehensive monitoring:

1. Model Performance Metrics

Track daily:

- Accuracy/Precision/Recall
- Prediction distribution (% positive predictions)
- Average confidence scores
- Error rates

Alert triggers:

- Accuracy drops >5%
- Prediction distribution shifts >10%
- Error rate increases >2x

2. Data Quality Monitoring

Track:

- Missing values (increased %)
- Feature distributions (shifted?)
- New categories (unseen in training)
- Input data volume (traffic patterns)

Example:

Training data: Average transaction = \$50

Current data: Average transaction = \$300

→ Alert: Data distribution changed!

→ Action: Retrain model

3. System Health Monitoring

Track:

- API latency (95th percentile)
- Error rates
- Throughput (requests/second)
- Resource usage (CPU, memory)

Dashboard Example:

Model Performance (Last 24h)	
Accuracy:	94.2% ✓
Avg Latency:	45ms ✓
Error Rate:	0.1% ✓
Requests:	1.2M ✓
Alerts	
⚠️ Feature X distribution	
shifted by 15%	
⚠️ Latency spike at 2pm	

4. Business Metrics Monitoring

- Track:
- Model predictions vs actual outcomes
 - Business KPIs influenced by model
 - Cost per prediction
 - ROI of the AI system

Example for churn prediction:

- Monitor:
- Customers predicted to churn → actually churned?
 - Retention rate for predicted high-risk customers
 - Cost of interventions vs value retained

Monitoring Tools:

- **Metrics:** Prometheus, Datadog, CloudWatch
- **Dashboards:** Grafana, Tableau
- **Alerts:** PagerDuty, Slack integration
- **ML-specific:** Weights & Biases, Neptune.ai

Warning Signs:

- "We'll add monitoring later"
- No defined metrics to track
- No alerting system
- Can't answer "How is the model performing?"

Why These Mistakes Are So Dangerous:

They all share a common problem: They surface LATE in the project lifecycle.

- Weeks 1-8: Building (everything seems fine)
- Week 9: Deployment
- Week 10: Oh no, costs are 100x expected ❌
- Week 12: System went down, no fallback ❌
- Week 16: Model degraded, we had no idea ❌

By then:

- Sunk costs (time and money)
- Pressure to keep broken system
- Painful to redesign
- Lost credibility

Prevention is cheaper than cure:

- Plan architecture upfront
 - Calculate costs early
 - Design fallbacks from day 1
 - Build monitoring before deployment
-

Slide 21: Group Exercise - Decision Making

Exercise Format:

This is a hands-on exercise to practice the decision-making framework you've learned. Work in small groups (3-4 people).

How It Works:

You will be given a business problem.

Your task:

1. **Decide on approach** (Rules / Classical ML / Traditional NLP / LLM)
2. **Identify constraints** (budget, latency, data, accuracy needs)
3. **Sketch high-level architecture** (components, data flow)
4. **Justify your decisions** (why this approach? why not others?)

Time: 15-20 minutes per problem **Focus:** Reasoning process, not technical implementation details

Sample Problems:

Problem 1: Restaurant Review Sentiment

Scenario: A restaurant chain wants to analyze customer reviews from Google and Yelp to understand sentiment.

Details:

- 50,000 reviews currently, growing 5,000/month
- Reviews are short (50-200 words)
- Need to classify: Positive, Neutral, or Negative
- Budget: \$500/month
- Results can be batch processed daily

Your Task:

1. What approach would you choose?

- ☐ Rules
- ☐ Classical ML
- ☐ Traditional NLP
- ☐ LLM

2. What are the key constraints?

- Budget:
- Latency:
- Data available:
- Accuracy needs:

3. Sketch architecture:

[Draw a simple diagram]

4. Why NOT the alternatives?

- Why not Rules?
- Why not LLM?

[Explain for each]

Discussion Points:

- Is this real-time or batch?
- How complex is the language?
- What's the cost per analysis?
- Can we use labeled data?

Problem 2: Real-Time Loan Approval

Scenario: A bank wants to automate loan approval decisions for personal loans up to \$10,000.

Details:

- Need instant decision (<500ms)
- Data: Income, credit score, employment history, debt ratio
- 10,000 applications/day
- Decisions must be explainable (regulatory requirement)
- Budget: \$2,000/month

Your Task:

1. What approach would you choose?

2. What are the critical constraints?

- Most critical:
- Second most:
- Third:

3. Architecture components:

- Data source:
- Processing:
- Model:
- Output:

4. Why this is better than LLM:

[List 3-5 reasons]

Discussion Points:

- What's the data structure?
- Why is latency critical?
- Why is explainability important?
- What's the regulatory context?

Problem 3: Legal Document Q&A

Scenario: A law firm wants to build a system where lawyers can ask questions about case documents.

Details:

- 1,000+ page legal documents
- Complex legal language and references
- Questions like: "What are the termination clauses?" or "Find all references to indemnification"
- 5-10 lawyers using it
- Budget: \$3,000/month
- Can tolerate 2-3 second response time

Your Task:

1. What approach would you choose?

2. Why is this different from previous problems?

3. Architecture design:

- How to handle long documents?
- How to find relevant sections?
- How to generate answers?

4. Cost-benefit analysis:

- Estimated cost:
- Alternative (manual) cost:
- ROI:

Discussion Points:

- Why can't Classical ML work here?
- Do we need an LLM? Why?
- Should we use RAG? Why?
- Is fine-tuning needed?

Problem 4: Product Recommendation

Scenario: An e-commerce site wants to show product recommendations on every product page.

Details:

- 1 million product views/day
- Must load recommendations in <100ms
- Data: User browsing history, purchase history, product metadata
- Budget: \$1,000/month
- Need personalized recommendations

Your Task:

1. What approach would you choose?

2. Key challenge:

3. Why this approach handles the challenge:

4. Architecture for scale:

- Preprocessing:
- Real-time component:
- Caching strategy:

Discussion Points:

- What's the scalability challenge?
- Why is latency so critical?
- Can we use pre-computation?
- Where does personalization happen?

What Good Answers Look Like:

Good reasoning includes:

- ✓ Identifies data type (structured vs unstructured)
- ✓ Recognizes critical constraints (latency, budget, explainability)
- ✓ Applies decision framework systematically
- ✓ Explains why NOT using other approaches
- ✓ Considers cost at scale
- ✓ Thinks about monitoring and fallbacks
- ✓ Balances accuracy with practical constraints

Poor reasoning:

- ✗ "Let's use LLM because it's powerful" (no analysis)
- ✗ Ignores constraints
- ✗ Doesn't consider alternatives
- ✗ No cost calculation
- ✗ Overcomplicates simple problems

Learning Objectives:

By completing these exercises, you should be able to:

1. Quickly categorize problems by type
2. Identify critical constraints
3. Apply decision framework systematically
4. Justify your choices with evidence
5. Recognize when simpler approaches are better

Remember: The goal is developing strong reasoning, not memorizing answers. Different valid approaches may exist—what matters is **how you think through the trade-offs**.

Slide 22: Key Takeaways

The Core Lessons from Days 8-9:

1. Choosing the Right Approach Matters More Than the Model

What this means:

- A simple model with the right approach beats a complex model with the wrong approach
- Spending time on problem framing is more valuable than hyperparameter tuning
- Architecture decisions have bigger impact than model selection

Example:

- ✗ Wrong: GPT-4 for fraud detection (99% accurate but \$50k/month)

✓ Right: XGBoost for fraud detection (97% accurate, \$500/month)

Winner: XGBoost (good enough accuracy, sustainable cost)

The Lesson: "Perfect" accuracy doesn't matter if the solution is unsustainable. Choose the simplest approach that meets business requirements.

Interview Impact: When asked "What model should we use?", the best answer starts with: "First, let's understand if this is even an ML problem, and what constraints we have..."

2. LLMs Are Powerful But Costly

LLMs Excel At:

- ✓ Complex language understanding
- ✓ Open-ended tasks
- ✓ Flexibility across different problems
- ✓ Few-shot learning (working with limited examples)

But They Come With High Costs:

- ✗ Expensive inference (\$0.01-0.03 per query)
- ✗ High latency (1-3 seconds)
- ✗ Infrastructure complexity
- ✗ Less interpretable

Cost Reality Check:

10,000 queries/day:
- Classical ML: \$100/month
- LLM API: \$6,000/month

That's 60x more expensive!

When to Use LLMs:

- Complex language tasks where nothing else works
- Budget allows for higher costs
- Can accept higher latency
- Value flexibility over efficiency

When NOT to Use LLMs:

- Structured data problems → Use Classical ML
- Simple text classification → Use Traditional NLP
- High-volume, low-latency needs → Use Classical ML
- Limited budget → Start with simpler approaches

Key Principle: "Use LLMs when they're necessary, not just because they're impressive."

3. Architecture Decisions Define Success

What This Means:

- How you build is as important as what you build
- Architecture determines scalability, cost, and maintainability
- Good architecture decisions prevent problems; bad ones create them

Critical Architecture Decisions:

1. Batch vs Real-time
2. Cloud API vs Self-hosted
3. Simple model vs Complex model
4. Monolithic vs Microservices
5. Caching strategy
6. Fallback plans

Example: Same Model, Different Architectures:

Architecture A: Direct API calls to LLM

- Cost: \$10,000/month
- No caching
- No fallback
- **Result:** Expensive, fragile

Architecture B: LLM with smart caching + fallback

- Cost: \$2,000/month (80% cache hit rate)
- Fallback to rules for common queries
- **Result:** Sustainable, reliable

The difference? Architecture, not the model itself.

Why Architecture Matters More Over Time:

Month 1: Model choice = 70% of success
Year 1: Architecture = 70% of success
Year 3: Architecture = 90% of success

As systems scale and evolve, architecture becomes increasingly critical.

4. Good AI Design Balances Trade-offs

The Reality: You cannot optimize for everything simultaneously.

The Four Dimensions:

1. **Accuracy** — How good are predictions?
2. **Cost** — How much does it cost?
3. **Latency** — How fast is it?
4. **Complexity** — How hard to build/maintain?

You must choose:

Want HIGH accuracy + LOW cost?
→ Accept higher complexity

Want LOW latency + HIGH accuracy?
→ Accept higher cost

Want LOW cost + LOW complexity?
→ Accept lower accuracy or higher latency

Real-World Balancing:

Startup (Limited Resources):

- Priority: Speed to market
- Accept: Higher operational costs (LLM API)
- Benefit: Launch fast, validate concept
- Plan: Optimize later if successful

Enterprise (High Scale):

- Priority: Low cost per transaction
- Accept: Higher complexity (self-hosted models)
- Benefit: Sustainable at millions of requests
- Plan: Invest in infrastructure and expertise

Healthcare (High Stakes):

- Priority: Explainability and accuracy
- Accept: Higher development complexity
- Benefit: Meet regulatory requirements
- Plan: Ensemble of interpretable models

The Framework for Balancing:

Step 1: Identify your constraints

Budget: \$X/month
Latency: <Y milliseconds
Accuracy: >Z%
Team size: N people
Timeline: T weeks

Step 2: Rank priorities

1. [Most critical]
2. [Important]
3. [Nice to have]
4. [Can sacrifice]

Step 3: Choose approach that satisfies top 2-3 priorities

Step 4: Accept trade-offs on lower priorities

Example Ranking:

E-commerce Recommendation:

1. Latency (<100ms) — Critical
2. Cost (<\$1k/month) — Critical
3. Accuracy (>80%) — Important
4. Complexity — Can sacrifice

Decision: Classical ML (collaborative filtering)
- Meets latency ✓
- Meets cost ✓
- Good enough accuracy ✓
- Higher complexity accepted ✓

The Master Principle:

"Build the simplest thing that works, then iterate based on real constraints."

Don't:

- Start with the most complex solution
- Optimize prematurely
- Use tools because they're trendy

Do:

- Start simple (rules, if possible)
- Add complexity only when justified
- Measure everything
- Iterate based on data

Success Checklist:

Before deploying any AI system, verify:

- ☐ Chosen simplest approach that meets requirements

- ☐ Calculated costs at expected scale
 - ☐ Designed for target latency
 - ☐ Planned monitoring strategy
 - ☐ Built fallback systems
 - ☐ Can explain decisions to stakeholders
 - ☐ Balanced trade-offs consciously
-

Slide 23: Homework / Practice

Assignment: Design a Complete AI Solution

This homework helps you practice the entire decision-making process. Choose ONE real-world problem and work through all the steps.

Your Task:

For one real business problem:

1. Choose AI approach (Rules / ML / NLP / LLM)
2. Justify why NOT alternatives
3. Estimate cost and latency
4. Sketch architecture

Deliverable Format:

Create a 1-2 page document with these sections:

Section 1: Problem Definition

Template:

Problem Statement:

[Describe the business problem in 2-3 sentences]

Business Context:

- Company/industry:
- Current solution (if any):
- Why AI is needed:

Success Criteria:

- Accuracy requirement:
- Latency requirement:
- Budget constraint:
- Other requirements:

Example:

Problem Statement:

A healthcare clinic wants to automatically categorize patient inquiries from email to route them to the right department (billing, appointments, medical questions, prescriptions).

Business Context:

- Company: Medium-sized clinic, 200 emails/day
- Current solution: Manual sorting by receptionist (2 hours/day)
- Why AI is needed: Free up receptionist time, faster routing

Success Criteria:

- Accuracy: >90% (some errors acceptable)
- Latency: <5 seconds (not real-time critical)
- Budget: <\$200/month
- Must be explainable to staff

Section 2: Approach Selection

Template:

Decision Framework:

1. Can rules solve this?

☐ Yes → Use rules

☐ No → Why not?

2. Structured data + labels?

☐ Yes → Consider Classical ML

☐ No → Why not?

3. Text-heavy, limited scope?

☐ Yes → Consider Traditional NLP

☐ No → Why not?

4. Open-ended language task?

☐ Yes → Consider LLM

☐ No → Why not?

Selected Approach: [Choose one]

Why this is best: [2-3 key reasons]

Example:

Decision Framework:

1. Rules? No

- Email inquiries too varied

- Can't write exhaustive rules

- Language is ambiguous

2. Classical ML? Maybe

- Could work with structured features

- Have historical labeled emails

- Fast classification

3. Traditional NLP? YES ✓

- Text classification task

- Limited categories (4-5)

- Don't need deep understanding

- Fast and cheap

4. LLM? Overkill

- Too expensive for volume

- Don't need conversation

- Simple categorization

Selected Approach: Traditional NLP (TF-IDF + Logistic Regression)

Why this is best:

1. Perfect for multi-class text classification

2. Fast (<50ms per email)

3. Within budget (can self-host)

4. Explainable (show key words for each category)

Section 3: Why NOT Alternatives

Template:

Rejected Approach #1: [Name]

Why rejected:

- Reason 1:
- Reason 2:
- Reason 3:

Rejected Approach #2: [Name]

Why rejected:

- Reason 1:
- Reason 2:
- Reason 3:

[Continue for all alternatives]

Example:

Rejected: Rules-Based System

Why rejected:

- Emails too varied: "want to reschedule", "change my appointment", "move my visit" all mean appointments
- Ambiguous language: "I have a question about my bill and also need to refill meds" → spans multiple categories
- Would need hundreds of rules, impossible to maintain

Rejected: LLM (GPT-4)

Why rejected:

- Cost: 200 emails/day × 30 days × \$0.02 = \$120/month just for API
- Latency: 2-3 seconds, unnecessary delay
- Overkill: Don't need deep understanding for simple categorization
- Budget better spent elsewhere

Rejected: Classical ML on structured features

Why rejected:

- Would need to manually engineer features (email length, keywords, etc.)
- Text embeddings (TF-IDF) automatically capture meaning
- NLP approach more appropriate for text data

Section 4: Cost and Latency Estimates

Template:

Cost Breakdown:

Development Costs (One-time):

- Data labeling: \$
- Model development: [hours] × \$[rate]
- Infrastructure setup: \$

Total Development: \$

Operational Costs (Monthly):

- Hosting/servers: \$
- API costs: \$
- Storage: \$
- Monitoring: \$
- Maintenance: [hours/month] × \$[rate]

Total Monthly: \$

Annual Cost: \$

Latency Estimate:

- Per prediction: [X] milliseconds
- How calculated: [Show math]
- Meets requirement? Yes/No

Scale Analysis:

- Current volume: [X] requests/day
- Expected growth: [Y]% per year
- Cost at 2x volume: \$
- Cost at 10x volume: \$

Example:

Cost Breakdown:

Development Costs (One-time):

- Data labeling: 1000 emails labeled → \$500
- Model development: 40 hours × \$50 = \$2,000
- Infrastructure setup: \$200

Total Development: \$2,700

Operational Costs (Monthly):

- Hosting: Small server \$50/month
- API costs: \$0 (self-hosted)
- Storage: \$5/month (email database)
- Monitoring: Basic logging \$0
- Maintenance: 2 hours/month × \$50 = \$100

Total Monthly: \$155/month

Annual Cost: \$2,700 + (\$155 × 12) = \$4,560

Latency Estimate:

- Per prediction: ~20 milliseconds
- How calculated:
 - TF-IDF vectorization: 5ms
 - Model inference: 10ms
 - Response formatting: 5ms
- Meets requirement (<5 seconds)? YES ✓

Scale Analysis:

- Current: 200 emails/day
- At 2x volume (400/day): Still \$155/month (same server)
- At 10x volume (2000/day): \$200/month (larger server)

Section 5: Architecture Sketch

Template:

High-Level Architecture:

- [Draw a simple diagram with:]
- Data sources
 - Processing pipeline
 - Model component
 - Storage
 - API/Interface
 - Monitoring

Components Description:

1. [Component name]
- Purpose:
- Technology:

2. [Next component]
- Purpose:
- Technology:

[Continue for each component]

Data Flow:

1. [Input arrives]
2. [Processing step 1]
3. [Processing step 2]
4. [Output delivered]

Example:

High-Level Architecture:

[Diagram:]

Email Inbox → Email Fetcher → Preprocessor → TF-IDF Vectorizer → Classifier Model → Category Label → Email Router → Department Inbox

Components Description:

1. Email Fetcher
- Purpose: Retrieve new emails every 5 minutes
- Technology: Python script, IMAP library
2. Preprocessor
- Purpose: Clean email text (remove signatures, HTML)
- Technology: BeautifulSoup, regex
3. TF-IDF Vectorizer
- Purpose: Convert text to numerical features
- Technology: scikit-learn TfidfVectorizer
4. Classifier Model
- Purpose: Predict category (Billing, Appointments, Medical, Rx)
- Technology: scikit-learn LogisticRegression
5. Email Router
- Purpose: Forward to appropriate department
- Technology: SMTP forwarding
6. Monitoring Dashboard
- Purpose: Track accuracy, volume, errors
- Technology: Simple PostgreSQL + Grafana

Data Flow:

1. New email arrives in general inbox
2. Fetcher retrieves email every 5 minutes (batch processing)
3. Preprocessor cleans text
4. TF-IDF converts to features
5. Model predicts category
6. System forwards to department
7. Log prediction for monitoring

Bonus Section: Monitoring and Iteration Plan

Template:

What to Monitor:

- Metric 1: [Name] — Track because: [reason]
- Metric 2: [Name] — Track because: [reason]
- Metric 3: [Name] — Track because: [reason]

Alert Conditions:

- If [metric] goes above/below [threshold] → [action]

Retraining Plan:

- Frequency: [How often?]
- Trigger: [What indicates need to retrain?]
- Process: [How will retraining work?]

Iteration Roadmap:

- Month 1: [Goal]
- Month 3: [Goal]
- Month 6: [Goal]

Example:

What to Monitor:

- Classification accuracy: Manual check of 20 random emails daily
- Category distribution: % of emails per category (detect unusual patterns)
- Confidence scores: Average model confidence (detect data drift)
- Processing time: Latency per email

Alert Conditions:

- If accuracy drops below 85% → Review recent errors, consider retraining
- If any category >50% of volume → Investigate possible system issue
- If latency >500ms → Check server load

Retraining Plan:

- Frequency: Monthly or when accuracy drops
- Trigger: Accuracy <85% or new types of inquiries observed
- Process: Collect new labeled emails (100-200), retrain model, A/B test

Iteration Roadmap:

- Month 1: Deploy basic system, establish baseline metrics
- Month 3: Add confidence threshold (manual review if confidence <70%)
- Month 6: Implement active learning (flag uncertain cases for human review)

Evaluation Criteria:

Your homework will be evaluated on:

1. **Problem Understanding** (20%)
 - Clearly defined problem
 - Realistic constraints
 - Appropriate success criteria
2. **Approach Selection** (30%)
 - Applied decision framework
 - Chose appropriate method
 - Strong justification
3. **Analysis of Alternatives** (20%)
 - Considered all options
 - Clear reasoning for rejection
 - Shows understanding of trade-offs
4. **Cost and Latency** (15%)
 - Realistic estimates
 - Showed calculations
 - Considered scaling
5. **Architecture Design** (15%)
 - Clear components
 - Logical data flow
 - Practical implementation

Good Work Demonstrates:

- ✓ Systematic thinking
- ✓ Understanding of trade-offs
- ✓ Practical considerations
- ✓ Clear communication

Common Mistakes to Avoid:

- ✗ Jumping to LLM without justification
- ✗ Ignoring cost/latency constraints

- ✗ Overly complex architecture
- ✗ Not considering alternatives
- ✗ Unrealistic estimates

Recommended Problems to Choose From:

Easy Level:

- Product review sentiment analysis
- Email spam detection
- Customer feedback categorization

Medium Level:

- Customer support ticket routing
- Resume screening system
- Content moderation system

Advanced Level:

- Legal document Q&A system
- Medical diagnosis support
- Financial fraud detection with explanation

Choose based on your comfort level and interest!

End of Day 8-9 Teaching Presentation

Summary of Learning Journey:

Day 8: You learned to distinguish between different AI approaches (Rules, Classical ML, Traditional NLP, LLMs) and when each is appropriate.

Day 9: You learned to design AI systems that balance real-world constraints (cost, latency, scalability, maintainability).

Combined: You've transitioned from model builders to AI solution designers who can make smart architectural decisions.

Next Steps:

1. Complete the homework assignment
2. Practice applying the decision framework to real problems
3. Start thinking about your own projects through this lens
4. Review the trade-off matrix when making decisions

Key Frameworks to Remember:

The Decision Framework:

Rules → Classical ML → Traditional NLP → LLM
(Start simple, add complexity only when necessary)

The Trade-off Principle:

You cannot optimize for everything.
Choose your battles wisely.

The Success Formula:

Right Approach + Right Architecture + Balanced Trade-offs = Production Success

Final Thought:

"The best AI solution is not the most advanced one—it's the one that actually ships, solves the problem, and stays within constraints."

Good luck with your AI solution design journey!